

3. Trade-offs

`2-key-design-decisions.md` covers individual choices; this section names the handful of overarching tensions those choices share.

3.1 Availability & throughput over strong consistency. Optimizing for: every write path is async end-to-end — Outbox + Debezium CDC decouples the DB write from the Kafka publish, and Kafka itself is at-least-once, not exactly-once (2.2, 2.5). No component blocks on another being available. Sacrificing: read-after-write consistency — a `GET /plans` right after `POST /plans` can briefly return stale data (1.3); duplicate messages/re-processing are expected, not exceptional, under crash-and-retry. Why: at tens of thousands of active plans with daily-in-aggregate executions, blocking on cross-component consistency would cap the whole pipeline’s throughput on its slowest part. Idempotency (2.5) makes at-least-once + eventual consistency safe rather than merely convenient.

3.2 Horizontal scalability & failure isolation over operational simplicity. Optimizing for: the Dispatch/Initiation split (2.4) and an independent Order-Status Consumer (1.7) mean no single component’s failure or slowness stalls the whole pipeline, and throughput scales by adding consumer instances. Sacrificing: component count — a naive “one service, one loop” design would be far simpler to deploy and debug than five Kafka topics, two consumer groups, and a separate Scheduler. More moving parts means more failure modes and monitoring surface. Why: the brief’s own scale context (daily-in-aggregate executions) makes the naive version a real bottleneck, not a hypothetical one — this is the one place complexity was spent deliberately rather than deferred.

3.3 Reuse & simplicity over optimal user experience (failure detection). Optimizing for: `PlanDeactivated`’s reactive trigger reuses the existing Order-Status Consumer flow (2.10) instead of a new reconciliation component; no cash/asset-availability pre-check (2.8) avoids duplicating Cash Domain logic. Sacrificing: a Plan can waste one full execution attempt — with no signal to the user until after it fails — before the system parks it. Why: proactive reconciliation is real, undesigned scope (2.10) — building it now means guessing at requirements we don’t have. Shipping the reactive path first, with the gap named, beats blocking MVP on a bigger unknown.

3.4 Domain decoupling over short-term efficiency. Optimizing for: zero synchronous cross-domain calls anywhere — order placement, cash sufficiency, and notification delivery are all discovered/handled via async Kafka events rather than direct queries to Cash, Trading, or Notification (2.8, 2.11). Sacrificing: latency in the unhappy path — an order that will obviously fail still goes through a full publish → Trading → reject → consume round trip instead of one synchronous check. Why: the brief mandates Kafka-only backend-to-backend communication and Cash allocation as out of scope — a synchronous shortcut here would mean quietly re-owning logic that belongs to another domain.

3.5 MVP-appropriate simplicity over completeness. Optimizing for: a single deployable service instead of split read/write services (2.1); full re-scan-on-crash instead of a dispatch checkpoint/cursor table (1.5); direct-produce instead of Outbox for the one component with no DB write to protect (2.2’s exception). Sacrificing: some efficiency headroom and architectural purity — a crash-and-retry redoes more work than a checkpointed version would. Why: each simplification has a named escape hatch (`7-mvp-post-mvp.md` 7.3) rather than being a dead end — chosen because current stated scale doesn’t yet justify the extra complexity, not because the scalable version is infeasible.